



البرمجة بلغة ++C

جنى الخطاب

Arrays and Strings

Data Types

- A data type is called simple if variables of that type can store only one value at a time.
- A structured data type is one in which each data item is a collection of other data items.

ال data type ينحكي عنه Simple اذا كان بخزن قيمة واحدة فقط بنفس الوقت مثل كل اللي اخذناهم قبل (int, char, ..) ، وينحكي عنه structured اذا العنصر الواحد هو عبارة عن مجموعة من العناصر كلهم يشتركوا في ال data type مثل ال arrays اللي رح نأخذ عنها في هذا التشابتر.

Arrays

- Array: a collection of a fixed number of components wherein all of the components have the same data type.

هي مجموعة من العناصر عددها ثابت وكل العناصر تشترك في نفس ال data type.

- In a one-dimensional array, the components are arranged in a list form.

في 1D array العناصر ترتب على شكل قائمة.

- Syntax for declaring a one-dimensional array:

```
dataType arrayName[intExp];
```

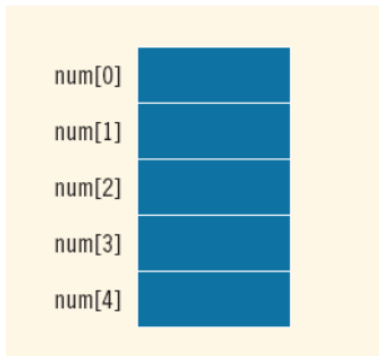
ال intExp هو عدد العناصر ويجب أن يكون positive integer.
أمثلة:

```
int arr [4];
```

```
char arr [6+7];
```

```
double marks [80/2];
```

- Example: `int num[5];`



يتم حجز مكان لها في ال memory كما في الشكل:

Accessing Array Components

للوصول إلى عنصر معين نعرف ترتيبه في ال array:

`arrayName[indexExp]`

where indexExp, called an index, is any expression whose value is a nonnegative integer.

بين ال [] square brackets (ال index قيمة ال indices تبدأ من صفر ثم واحد اثنين الخ).

- Index value specifies the position of the component in the array

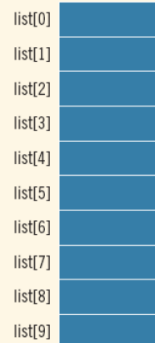
قيمة هذا ال index تحدد موضع العنصر في ال array.

- The array index always starts at 0.

قيمة ال index دائماً تبدأ من الصفر يعني index العدد الأول هو صفر و index العدد الثاني واحد وهكذا.

index=position of the component – 1

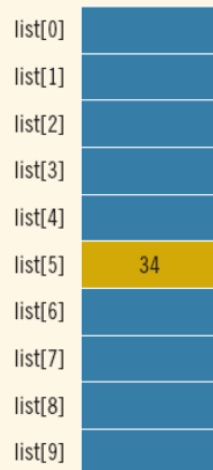
```
int list[10];
```



list[0]	
list[1]	
list[2]	
list[3]	
list[4]	
list[5]	
list[6]	
list[7]	
list[8]	
list[9]	

عند تعريف `int list[10]` يتم حجز 10 مواضع
متتاليات من النوع `int`. لاحظ أن عدد ال `indices` يقل
عن ال `array size` بمقدار واحد.

```
list[5] = 34;
```



list[0]	
list[1]	
list[2]	
list[3]	
list[4]	
list[5]	34
list[6]	
list[7]	
list[8]	
list[9]	

في هذا المثال تم اعطاء قيمة
ابتدائية للعنصر السادس
(بال `index` الخامس).

الأثر الجميل يعمل بصمت

```
list[3] = 10;
list[6] = 35;
list[5] = list[3] + list[6];
```

list[0]	
list[1]	
list[2]	
list[3]	10
list[4]	
list[5]	45
list[6]	35
list[7]	
list[8]	
list[9]	

في هذا المثال، تم إعطاء قيمة ابتدائية لعنصر باستخدام قيم عناصر أخرى.

طريقتين لتحديد ال : array size

1. تعبير رياضي يحول الى : positive integer

```
int arr [6/5];
```

2. باستخدام (int. constant identifier)

```
const int array_size = 4;
```

```
int arr[array_size];
```

```
int arraySize;
```

```
cout << "Enter the size of the array: ";
cin >> arraySize;
cout << endl;
```

```
int list[arraySize];
```

نعطي array size بهذه الطريقة خطأ؛ أولاً لأنه variable ليس constant ثانياً لأن ال array size يجب أن يكون محدد عند تعريف array.

Processing One-Dimensional Arrays

- Some basic operations performed on a onedimensional array are:

1- Initializing

2- Inputting data

3- Outputting data stored in an array

4- Finding the largest and/or smallest element

- Each operation requires ability to step through the elements of the array, easily accomplished by a loop.

رح نتعلم عمليات تطبيقها على ال : array نعطي قيمة ابتدائية للعناصر ، نأخذ العناصر ك input ، نطبع العناصر المخزنة، إيجاد أكبر وأصغر عنصر، كل عملية من هذول بتحتاج انه نوصل للعناصر في ال array من خلال استخدام for loop

عشان نوصل لكل العناصر في array معينة:

```
for (i=0; i<array_size; i++)
```

1.inputting data:

```
int list [100];
```

```
for (i=0; i<100; i++)
```

```
cin>>list[i];
```

2.initializing an array:

```
double arr [30];
```

```
for (i=0; i<30; i++)
```

```
arr[i]=1.5;
```

3.Printing an array:

```
int arr[5];  
for (i=0; i<5; i++)  
cout<<arr[i]<<" ";
```

4.finding the sum and average of all elements:

```
int arr[5];  
int sum = 0;  
for (i=0; i<5; i++)  
sum+=arr[i];  
double avg = sum/5;
```

5.finding the largest element:

```
int max_index = 0;  
int arr[5];  
for (i=0; i<5; i++)  
if ( arr[max_index] < arr[i])  
max_index = i;  
cout<<" The largest element is : "<<arr[max_index];
```

Array Index Out of Bounds

*إذا في array حجمها 5 يعني ال indices الخاصين فيها هم 0,1,2,3,4: أي index غير هيك بنقول انه index is out of bounds.

Array Initialization During Declaration

نقدر نعطي قيمة ابتدائية للعناصر كلها خلال تعريف ال array ، في هذه الحالة لا داعي أن نحدد حجم ال array نترك
الاقواس فارغة ويتم حساب الحجم حسب عدد العناصر اللي ادخلته كالاتي:

```
double sales[] = {12.25, 32.50, 16.90, 23, 45.68};
```

مثلا حجم هذه ال array يساوي 5.

```
int list[] = {5, 6, 3};
```

في هذا المثال حجم ال array يساوي 3.

Partial Initialization of Arrays During Declaration

لما تحدد ال array size خلال التعريف وتعطي قيم ابتدائية ويكون عددها أقل من حجم ال array العناصر الباقية كلها تكون قيمتها صفر. by default

مثال:

```
int list[10] = {0};
```

في هذا المثال يكون list[0]=0 وباقي العناصر صفر. by default

```
int list[10] = {8, 5, 12};
```

في هذا المثال يكون list[0] = 8, list[1] = 5, list[2] = 12 وباقي العناصر صفر. by default

Some Restrictions on Array Processing

مثلا لو عندي two arrays كالاتي:

```
int myList[5] = {0, 4, 8, 12, 16}; //Line 1
int yourList[5]; //Line 2
```

لو بدنا نساوي عناصرهم ببعض غلط نعمل
الاتي:

```
yourList = myList; //illegal
```

***it is an error**

يجب أن نساوي كل عنصر بالعنصر المقابل به كالاتي:

```
for (int index = 0; index < 5; index ++)  
    yourList[index] = myList[index];
```

كل الجمل الآتية خطأ أيضا:

cin>>myList; *it is an error

cout<<yourList; *it is not an error

if (yourList < myList) *it is not an error

المكتوب عندهم انهم مش ايرور برضه ما بيعطوا النتيجة المطلوبة.

الحل الوحيد لكل هذه الجمل الخطأ أن نستخدم for loop يمر على كل العناصر.

Arrays as Parameters to Functions

- Arrays are passed by reference only.

إذا اردنا إرسال array الى function ترسل كـ reference parameter.

```
void funcArrayAsParam(int listOne[], double listTwo[])  
{  
    .  
    .  
    .  
}
```

إذا لاحظتوا لا نستخدم & لأنه by default يعتبرها reference ولا يتم تحديد ال size وإذا تم تحديده الكومبايلر بعمل عنه skip.

Constant Arrays as Formal Parameters

```
void printArray const int list[], int listSize)
{
    int index;

    for (index = 0; index < listSize; index++)
        cout << list[index] << " ";
}
```

طبعاً في حال اردنا أن لا يطرأ أي تغييرات على ال array داخل ال function نستطيع أن نجعلها constant كالمثال اعلاه.

Base Address of an Array and Array in Computer Memory

- The base address of an array is the address, or memory location of the first array component.
- When we pass an array as a parameter, the base address of the actual array is passed to the formal parameter.

ال base address لل array هو ال memory location (أول عنصر) يعني لل index صفر)، ولما نرسل array الى function يتم ارسال هذا ال base address.

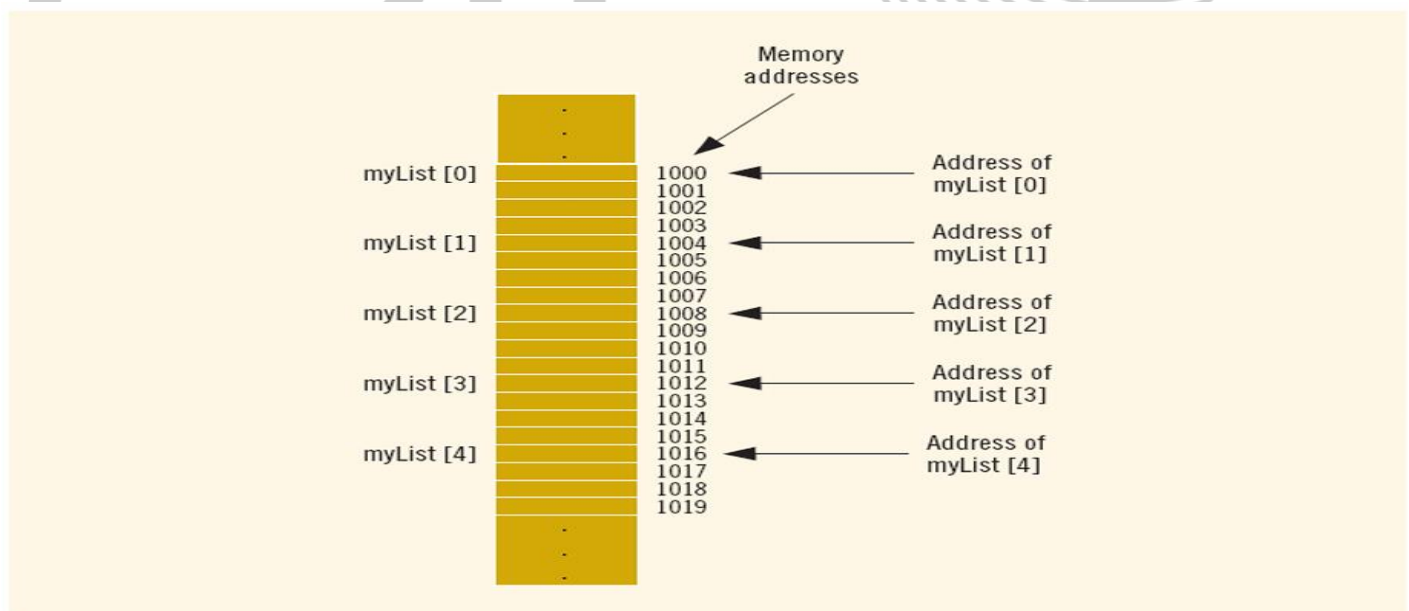


FIGURE 9-6 Array myList and the addresses of its components

Functions Cannot Return a Value of the Type Array

- C++ does not allow functions to return a value of the type array.

ما بنقدر نرجع array من function ، عشان هيك ال arrays ما بنرسلوا الـ by reference.

Two-Dimensional Arrays

- Two-dimensional array: collection of a fixed number of components (of the same type) arranged in two dimensions.

– Sometimes called matrices or tables.

الـ 2D array هي مجموعة من العناصر مرتبة في اتجاهين (سطور وأعمدة (ممكن أن تسمى) matrix مصفوفة (أو جدول).

- Declaration syntax:

```
dataType arrayName[intExp1][intExp2];
```

الفرق عن الـ 1D array نضع two square brackets واحد لحجم الأسطر والثاني لحجم الأعمدة بالترتيب) طبعا كل شروط الـ 1D تنطبق عليه، انه يكون positive integer و constant الخ).

```
double sales[10][5];
```

sales	[0]	[1]	[2]	[3]	[4]
[0]					
[1]					
[2]					
[3]					
[4]					
[5]					
[6]					
[7]					
[8]					
[9]					

في هذا المثال تعريف 2D array حجمها 10x5 ، (عشر أسطر وخمسة أعمدة).

FIGURE 9-8 Two-dimensional array sales

Accessing Array Components

- Syntax:

```
arrayName[indexExp1][indexExp2]
```

لنوصل الى عنصر معين نفس ما كنا نعمل ب 1D array الفرق انه نضيف أقواس للاتجاه الثاني وهو الأعمدة.

```
sales[5][3] = 25.75;
```

sales	[0]	[1]	[2]	[3]	[4]
[0]					
[1]					
[2]					
[3]					
[4]					
[5]				25.75	
[6]					
[7]					
[8]					
[9]					

sales [5] [3]

FIGURE 9-9 sales[5][3]

Two-Dimensional Array Initialization During Declaration

```
int board[4][3] = {{2, 3, 1},  
                  {15, 25, 13},  
                  {20, 4, 7},  
                  {11, 18, 14}};
```

نقدر نعطي قيمة ابتدائية للعناصر خلال التعريف كل سطر بين { } ونفصل السطور بفاصلة. إذا كان تحديد العناصر أقل من حجمها يكون قيمة الباقي صفر by default كما في ال 1D array.

Processing Two-Dimensional Arrays

- Ways to process a two-dimensional array:
 - Process the entire array
 - Process a particular row of the array, called row processing
 - Process a particular column of the array, called column processing.

يمكن نعمل عمليات على ال array كلها ممكن على سطر أو عمود معين.
مثال:

```
const int NUMBER_OF_ROWS = 7;    //This can be set to any number.
const int NUMBER_OF_COLUMNS = 6; //This can be set to any number.

int matrix[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];
int row;
int col;
int sum;
int largest;
int temp;
```

matrix	[0]	[1]	[2]	[3]	[4]	[5]
[0]						
[1]						
[2]						
[3]						
[4]						
[5]						
[6]						

FIGURE 9-13 Two-dimensional array matrix

```
for (row = 0; row < NUMBER_OF_ROWS; row++)
    for (col = 0; col < NUMBER_OF_COLUMNS; col++)
        matrix[row][col] = 0;
```

اعطاء قيمة ابتدائية لل array كلها:

```
row = 4;
for (col = 0; col < NUMBER_OF_COLUMNS; col++)
    matrix[row][col] = 0;
```

اعطاء قيمة ابتدائية لسطر معين) في هذا المثال (5 لاحظوا ان index الاسطر ثابت و index الأعمدة يتغير.

Print

```
for (row = 0; row < NUMBER_OF_ROWS; row++)
{
    for (col = 0; col < NUMBER_OF_COLUMNS; col++)
        cout << setw(5) << matrix[row][col] << " ";

    cout << endl;
}
```

طباعة عناصر ال array :

*ملاحظة setw: هو function نستخدمه لتظهر العناصر بشكل أرتب.

1	23	456	7890
12	234	56	7

مع استخدامه

1	23	456	7890
12	234	56	7

بدون استخدامه

Input

```
for (row = 0; row < NUMBER_OF_ROWS; row++)
    for (col = 0; col < NUMBER_OF_COLUMNS; col++)
        cin >> matrix[row][col];
```

لإدخال عناصر ال 2D array :

Sum by Row

```
sum = 0;
row = 4;
for (col = 0; col < NUMBER_OF_COLUMNS; col++)
    sum = sum + matrix[row][col];
```

جمع عناصر سطر معين) في هذا المثال.(5

```
//Sum of each individual row
for (row = 0; row < NUMBER_OF_ROWS; row++)
{
    sum = 0;
    for (col = 0; col < NUMBER_OF_COLUMNS; col++)
        sum = sum + matrix[row][col];

    cout << "Sum of row " << row + 1 << " = " << sum << endl;
}
```

في هذا المثال يتم حساب مجموع كل سطر على حدى.

Sum by Column

```
//Sum of each individual column
for (col = 0; col < NUMBER_OF_COLUMNS; col++)
{
    sum = 0;
    for (row = 0; row < NUMBER_OF_ROWS; row++)
        sum = sum + matrix[row][col];

    cout << "Sum of column " << col + 1 << " = " << sum << endl;
}
```

في هذا المثال يتم حساب مجموع عناصر كل عمود على حدى.

Largest Element in Each Row and Each Column

```
//Largest element in each row
for (row = 0; row < NUMBER_OF_ROWS; row++)
{
    largest = matrix[row][0]; //Assume that the first element
                             //of the row is the largest.
    for (col = 1; col < NUMBER_OF_COLUMNS; col++)
        if (largest < matrix[row][col])
            largest = matrix[row][col];

    cout << "The largest element in row " << row + 1 << " = " << largest << endl;
}
```

في هذا المثال يتم حساب أكبر عنصر في كل سطر (على افتراض أن أول عنصر في السطر هو الأكبر ثم مقارنته مع باقي العناصر في نفس السطر).

```

//Largest element in each column
for (col = 0; col < NUMBER_OF_COLUMNS; col++)
{
    largest = matrix[0][col]; //Assume that the first element
                             //of the column is the largest.
    for (row = 1; row < NUMBER_OF_ROWS; row++)
        if (largest < matrix[row][col])
            largest = matrix[row][col];

    cout << "The largest element in column " << col + 1
         << " = " << largest << endl;
}

```

في هذا المثال يتم حساب
أكبر عنصر في كل عمود
(نفس مبدأ المثال السابق).

Problem-Solving Part

1. Write a C++ program that contains a function arraySort that takes an integer array and its size as parameters and sorts the array in ascending order.
2. Write a C++ program that inputs the size of columns of 2D array (with two rows) and input the array then define a function that swaps the two rows and another function to calculate the sum of the diagonal.
3. Write a C++ program that takes an integer matrix of size 4x4 as input from the user, then calculates and prints the sum of each row.
4. Write a C++ program that contains a function findMax that takes an integer array and its size as parameters and returns the maximum number in the array.

```
#include <iostream>
using namespace std;

void arraySort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}

int main() {
    int size;

    cout << "Enter the size of the array: ";
    cin >> size;
```



الأثر

```

int arr[size]; // using variable length array (VLA) if supported by compiler

cout << "Enter " << size << " elements of the array: ";
for (int i = 0; i < size; i++) {
    cin >> arr[i];
}

cout << "Original array: ";
for (int i = 0; i < size; i++) {
    cout << arr[i] << " ";
}
cout << endl;

arraySort(arr, size);

cout << "Sorted array: ";
for (int i = 0; i < size; i++) {
    cout << arr[i] << " ";
}
cout << endl;

return 0;
}

```

```

#include <iostream>
using namespace std;

// Function to calculate sum of diagonal elements in a 2D array
int diagonalSum(int arr[][3], int rows, int columns) {
    int sum = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            if (i == j)
                sum += arr[i][j];
        }
    }
    return sum;
}

int main() {
    int rows, columns;

    // Input dimensions of the array
    cout << "Enter the number of rows and columns respectively separated by a space: ";
    cin >> rows >> columns;

    // Declare a 2D array with constant size
    int arr[rows][3]; // Assuming second dimension is 3 for example

    // Input array elements
    cout << "Enter elements of the array:\n";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            cin >> arr[i][j];
        }
    }

    // Calculate and display diagonal sum
    cout << "The sum of the diagonal elements is: " << diagonalSum(arr, rows, columns) << endl;
    return 0;
}

```

```
#include <iostream>
using namespace std;
int main() {

    int arr[4][4];
    for (int i=0;i<4;i++)
    {
        for (int j=0;j<4;j++)
        {
            cin>>arr[i][j];
        }
    }

    for (int i=0;i<4;i++)
    {
        int sum=0;
        for (int j=0;j<4;j++){
            sum+=arr[i][j];
        }

        cout<<"Row "<<i+1<<" Sum: "<<sum;|
    }
    return 0;
}
```

4.

الأثر الجميل يعمل بصمت

```
#include <iostream>
using namespace std;

int Findmax(int arr[], int size)
{
    int MAX = arr[0];
    for (int i=1;i<size;i++)
    {
        if ( arr[i]>MAX)
            MAX = arr[i];
    }
    return MAX;
}

int main() {
    int size;
    cout << "Enter the size of the array: ";
    cin >> size;

    int arr[size];

    cout << "Enter " << size << " elements: ";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

    cout << "Max in array: " << Findmax(arr, size);

    return 0;
}
```

الأثر الجميل يعمل بصمت